

Custom FFT Hardware Implementation: Architecture, Verification, and Performance

EE24BTECH11048
Nithin K

1 Architecture and Implementation

The custom hardware design implements an 8-point, Radix-2 Fast Fourier Transform (FFT) utilizing a pipelined architecture. The system is split into the core processing module and an automated verification testbench.

1.1 Pipeline Architecture

The core processing module (`fft8_pipeline`) utilizes a Decimation-In-Time (DIT) algorithm, functioning through the following sequence:

- **Input Buffering and Bit-Reversal:** Serial input samples (`din_re`, `din_im`) are buffered into a shift register until an 8-point frame is complete. Upon filling, the data is loaded into parallel registers using bit-reversed indexing to prepare for the DIT butterfly stages.
- **Pipelined Butterfly Stages:** The processing is divided into three distinct synchronous stages. Stage 1 executes parallel 2-point butterflies. Stage 2 executes 4-point butterflies. Stage 3 completes the transformation with 8-point butterflies, resolving the final frequency bins.
- **Fixed-Point Arithmetic:** Twiddle factor multiplications in the final stage are handled by custom fixed-point functions (`mul_re` and `mul_im`), utilizing arithmetic right shifts to maintain scaling without relying on heavy floating-point logic or vendor DSP blocks. Constants for the twiddle factors are pre-calculated and parameterized.

1.2 Testbench and Verification Logic

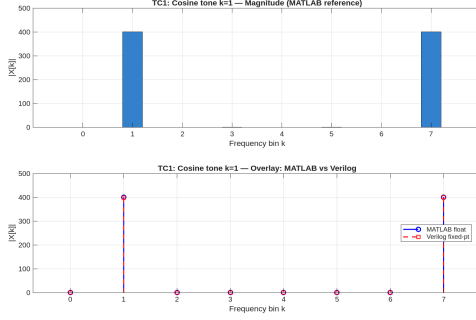
The testbench (`fft8_tb`) is designed for automated, self-checking verification.

- **Stimulus Generation:** A task-based structure is used to clock in five distinct discrete test signals: a cosine tone, an impulse, a DC signal, a complex exponential, and a two-tone combination.
- **Self-Checking Assertions:** Upon assertion of the `dout_valid` flag, the testbench automatically checks the magnitude of the 24-bit output bins against the mathematically expected results, outputting a clear PASS/FAIL metric for each test case directly into the simulation console.

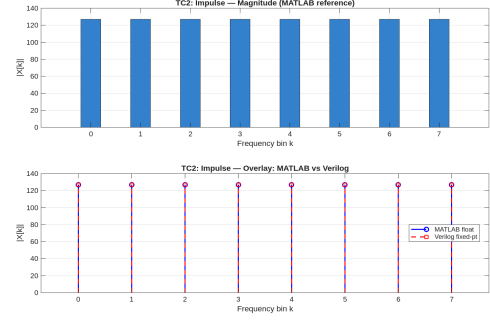
2 Functional Verification

The custom Verilog fixed-point Fast Fourier Transform (FFT) module was verified against a MATLAB floating-point reference model. Various test signals were applied to validate the

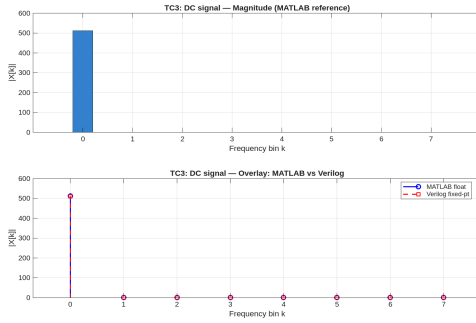
accuracy of the frequency bins. As shown in the overlay plots, the output of the custom fixed-point implementation aligns with the MATLAB reference across the discrete test cases.



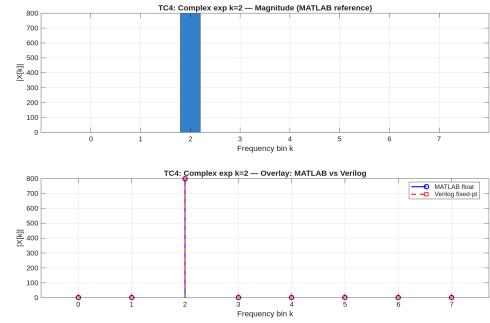
(a) TC1: Cosine tone ($k = 1$)



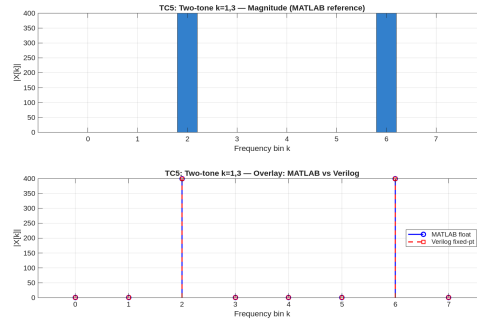
(b) TC2: Impulse



(c) TC3: DC Signal



(d) TC4: Complex exponential ($k = 2$)



(e) TC5: Two-tone ($k = 1, 3$)

Figure 1: Magnitude spectrum overlays comparing MATLAB floating-point reference vs. Custom Verilog fixed-point implementation.

3 Synthesis Results and Performance

The custom design was synthesized and compared against a standard vendor FFT IP core to document resource utilization and timing metrics. Table 1 summarizes the resource allocation and maximum operating frequency (F_{max}) extracted from the respective compilation reports.

Table 1: Performance and Resource Metrics: Custom Verilog vs. Vendor IP

Metric	Custom Verilog	Vendor IP Core
Maximum Frequency (F_{max})	238.55 MHz	168.21 MHz
Combinational ALUTs	147	761
Dedicated Logic Registers	446	1,240
Total Block Memory Bits	0	40,960
DSP Blocks	0	2

3.1 Throughput Analysis

Based on the input logic of the custom `fft8_pipeline` module, the shift registers require 8 continuous clock cycles to buffer an 8-point data frame. An additional clock cycle is consumed to trigger the `full` condition and latch the buffered data into the DIT pipeline. This creates a state machine footprint of 9 clock cycles per 8-point data frame.

Using the post-synthesis maximum operating frequency (F_{max}) of 238.55 MHz, the theoretical maximum throughput for continuous streaming is calculated as follows:

$$\text{Transform Rate} = \frac{F_{max}}{9 \text{ cycles/frame}} \approx 26.50 \text{ Mega-transforms/sec}$$

$$\text{Sample Throughput} = 26.50 \text{ MT/s} \times 8 \text{ samples/frame} \approx 212.04 \text{ MSPS}$$

This demonstrates that the custom logic path maintains a high-speed data ingestion rate of approximately 212.04 Mega-samples per second while utilizing a minimal logic footprint.